PERFORMANCE · Week 7 · Fri

Web Performance Optimization

Fast sites win — 53% of users leave after 3 seconds

K0126002 · Gachon University · 2026 Fall · Dr. Toshpulatov

Why Performance Matters — The Numbers

53%

of mobile users abandon a site that takes more than 3 seconds to load

1s

delay = 7% drop in conversions (Amazon)

100ms

delay = 1% revenue drop (Google)

3s

the threshold where 53% leave (mobile)

0.1s

feels "instant" to users — the goal

Google Lighthouse — 5 Audit Categories

Performance

0–100

LCP, FID, CLS — Core Web Vitals and more

Accessibility

0–100

Contrast, labels, ARIA

Best Practices

0–100

HTTPS, no console errors, image aspect ratios

SEO

0–100

Meta tags, robots.txt, mobile-friendly

PWA

Pass/Fail

Service worker, manifest — optional

QUICK QUIZ

Google Lighthouse measures performance with scores from 0–100. A score of 90+ means:

A The site loads in under 1 second

B The site passes all performance checks (Good category)

C The site's load time is average

D The site has no JavaScript errors

Discuss — then we'll reveal the answer

✅ ANSWER

Google Lighthouse measures performance with scores from 0–100. A score of 90+ means:

A The site loads in under 1 second

B The site passes all performance checks (Good category)

C The site's load time is average

D The site has no JavaScript errors

Lighthouse grades by weighted metrics (LCP, FID, CLS, etc.). 90–100 = Good. It does NOT guarantee sub-1s load time.

Network Tab in DevTools

```
// Open: F12 > Network > reload page
// Key things to look for:
// 1. Large files (images > 200KB)
//    Sort by Size column
// 2. Blocking requests at top
//    CSS/JS blocking the render
// 3. Long chain of serial requests
//    Each waits for the previous
// 4. 304 Not Modified = cached
//    (good! browser reused it)
// 5. Duplicate requests
//    Same file loaded twice
```

Reading the Waterfall

Waterfall
Timeline of every request

Size
Compressed vs original bytes

Time
How long each request took

Initiator
What triggered the request

Status
200, 304, 404, 500...

Image Optimization — The Biggest Win

Format	Best Use	Savings	When to Use
JPEG	Photos, complex images	Smallest for photos	Always for photographs
PNG	Transparency, logos	Lossless but larger	When transparency needed
WebP	Photos + transparency	25-35% vs JPEG/PNG	Modern browsers (2020+)
SVG	Icons, logos, charts	Infinitely scalable	Vector art, always crisp

Tools: Squoosh (squoosh.app), TinyPNG — compress before you ship

Key rule: serve images at the size they are DISPLAYED, not the original camera size.

loading="lazy" on Images

```
<!-- All images load on page load ❌ -->
<img src="photo1.jpg" alt="Team photo">
<img src="photo2.jpg" alt="Campus">
<img src="photo3.jpg" alt="Library">

<!-- Only visible images load first ✅ -->
<!-- Hero: always eager (above fold) -->
<img src="hero.jpg" alt="Welcome"
     loading="eager"
     width="1200" height="600">

<!-- Card images: lazy (below fold) -->
<img src="photo1.jpg" alt="Team"
     loading="lazy"
     width="400" height="300">
<!-- width/height prevent CLS -->
```

Defer Off-Screen Images

loading="lazy"

Load when near viewport

loading="eager"

Load immediately (default)

Threshold

Browser decides ~500-1000px

Above-fold

Never lazy-load hero images

width + height

Set both to prevent CLS

QUICK QUIZ

Which HTML attribute tells the browser to load an image only when it's near the viewport?

A src="lazy"

B defer="true"

C loading="lazy"

D async="true"

Discuss — then we'll reveal the answer

✅ ANSWER

Which HTML attribute tells the browser to load an image only when it's near the viewport?

A src="lazy"

B defer="true"

C loading="lazy"

D async="true"

loading="lazy" is a native HTML attribute — no JavaScript needed. defer and async apply to scripts, not images.

Minification — Remove the Whitespace

BEFORE (3.2 KB)

```
/* Button styles */
button {
  background-color: #1A2540;
  color: #FFFFFF;
  padding: 12px 24px;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;
}
/* Hover effect */
button:hover {
  background-color: #F7DF1E;
  color: #1A2540;
}
```

AFTER (0.6 KB) — 81% smaller

```
button{background-color:#1A2540;
color:#fff;padding:12px 24px;
border:none;border-radius:4px;
font-size:16px;cursor:pointer}
button:hover{background-color:
#F7DF1E;color:#1A2540}
```

QUICK QUIZ

What does JavaScript minification do?

A Splits code into smaller files

B Removes whitespace, comments and shortens variable names

C Compresses images in the page

D Moves scripts to a CDN

Discuss — then we'll reveal the answer

✅ ANSWER

What does JavaScript minification do?

A Splits code into smaller files

B Removes whitespace, comments and shortens variable names

C Compresses images in the page

D Moves scripts to a CDN

Minification strips characters humans need (spacing, long names) but the browser does not. It shrinks file size without changing behavior.

The defer Attribute

```
<!-- ❌ Blocks parsing — browser stops
     reading HTML until script runs -->
<script src="app.js"></script>

<!-- ✅ defer: parse HTML + download JS
     simultaneously. Runs after HTML parsed -->
<script src="app.js" defer></script>

<!-- async: runs as soon as downloaded
     (order not guaranteed) -->
<script src="analytics.js" async></script>

<!-- Best practice: all scripts in <head>
     with defer — NOT at bottom of <body> -->
```

Load JS Without Blocking

Normal <script>

Blocks HTML parsing

defer

Download parallel, run after HTML

async

Download parallel, run ASAP

type="module"

Deferred automatically

Rule of thumb

Use defer for most scripts

QUICK QUIZ

<script src="app.js" defer> — when does the browser execute app.js?

A Immediately, blocking HTML parsing

B In parallel with HTML parsing, then after HTML is fully parsed

C After all images load

D Only when a user event fires

Discuss — then we'll reveal the answer

✅ ANSWER

<script src="app.js" defer> — when does the browser execute app.js?

A Immediately, blocking HTML parsing

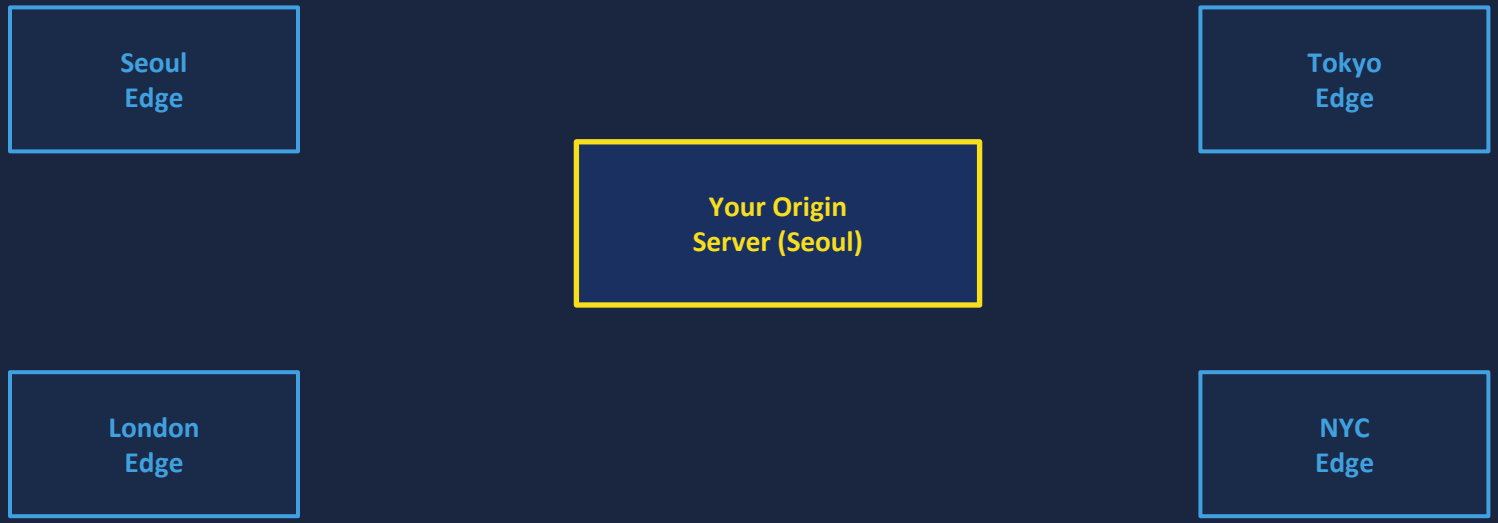
B In parallel with HTML parsing, then after HTML is fully parsed

C After all images load

D Only when a user event fires

defer downloads in parallel (non-blocking) and executes after the DOM is ready, preserving script order. async also downloads in parallel but runs immediately when downloaded.

CDN — Content Delivery Network



Korean user → Seoul copy | Japanese user → Tokyo copy

- ⚡ Files served from a nearby server — lower latency
- 📦 Static assets (images, CSS, JS) cached at the edge
- 💰 Free CDNs for libraries: cdnjs.com, jsDelivr.com
- 🌍 Same URL works worldwide — CDN routes the request

QUICK QUIZ

CDN stands for:

A Core Data Network

B Cached DNS Node

C Content Delivery Network

D Central Domain Node

Discuss — then we'll reveal the answer

✅ ANSWER

CDN stands for:

A Core Data Network

B Cached DNS Node

C Content Delivery Network

D Central Domain Node

A CDN stores copies of your static files on servers worldwide. Users download from the closest server, reducing latency.

Browser Caching

```
# Cache JS/CSS for 1 year
# (change filename on update)
Cache-Control: public, max-age=31536000

# Cache HTML for 5 minutes
Cache-Control: public, max-age=300

# Never cache (live data)
Cache-Control: no-store

# Browser asks "is this stale?"
# Server says NO — browser uses cache
HTTP/1.1 304 Not Modified

# See in Network tab:
# Status=304, Size="(disk cache)"
```

Cache-Control & 304

Cache-Control

How long to keep the file

max-age=31536000

Cache 1 year (immutable)

304 Not Modified

Server says: use your cache!

ETag

File fingerprint for comparison

Cache busting

filename.v2.js on change

Above-the-Fold — Prioritize Visible Content

1

Critical CSS inline

Put essential styles in <style> in <head> so the first render has no extra HTTP round trip.

2

No render-blocking JS

Use defer on scripts. Even tiny blocking JS delays the first paint.

3

Preload key assets

<link rel="preload" href="hero.jpg"> tells the browser to fetch it early.

4

Font display: swap

CSS: font-display: swap; shows text immediately, swaps font when loaded.

Performance Checklist — 10 Quick Wins

- ✅ Compress all images — use WebP where possible
- ✅ Use a CDN for third-party libraries
- ✅ Set width and height on all <img> tags
- ✅ Enable GZIP/Brotli on your server
- ✅ Add loading="lazy" on all below-fold images
- ✅ Set Cache-Control headers for static assets
- ✅ Add defer to all <script> tags
- ✅ Preload your hero image and key fonts
- ✅ Minify your CSS and JS files
- ✅ Run Lighthouse — target Performance 90+

Students should aim to check off all 10 before submitting their final project.

Summary & Homework

Images

WebP, correct size, loading="lazy"

Minification

Smaller CSS/JS files = faster load

defer

Non-blocking script loading

CDN

Serve from edge nodes worldwide

Caching

Cache-Control, 304 Not Modified

Lighthouse

Measure, fix, measure again

Lecture notes: Performance_v2.pdf
_StudyGuide: Web_Performance_Optimization

Practical guides: index.html

HOMEWORK

Optimize your personal site: (1) Run Lighthouse, screenshot your Performance score. (2) Apply at least 3 optimizations from today. (3) Re-run Lighthouse, screenshot new score. Post both screenshots to your page with a 2-sentence description of what you changed.

https://kahlen2008.github.io/webprogramming/week%209/week%209_2/hw-lighthouse-performance.html